

lab4实验报告

思考题

Thinking 4.1

Q: 思考并回答下面问题

1. 内核在保存现场的时候是如何避免破坏通用寄存器的？
2. 系统陷入内核调用后可以直接从当时的\$a0-\$a3参数寄存器中得到用户调用msyscall留下的信息吗？
3. 我们是怎么做到让sys开头的函数“认为”我们提供了和用户调用msyscall时同样的参数的？
4. 内核处理系统调用的过程对Trapframe做了哪些更改？这种修改对应的用户态的变化是什么？

A:

1. 内核使用SAVE_ALL宏保留现场，并在保存现场时只使用了k0寄存器，在恢复时仅使用了v0和v1寄存器。
2. 可以。因为内核在陷入内核、保存现场的过程中，寄存器 a3中的值都没有被破坏。
3. 用户在调用msyscall以及陷入内核时，a0-a3寄存器的值并不会发生变化，用户栈的内容也会被拷贝到内核栈，因此可以获取相同的参数。
4. 将Trapframe中的epc+4,并且将返回值写到v0寄存器中。用户态能够从v0寄存器中直接得到返回值，并不会再执行发生异常的指令。

Thinking 4.2

Q:

思考 env_id2env 函数: 为什么 env_id2env 中需要判断 e->env_id != env_id的情况？如果没有这步判断会发生什么情况？

A:

理论上不可能出现这种情况，但是在我们完成一个进程并回收了其资源时，它会进入env_free_list中，在我们还未给它重新赋予新的env_id时就可能出现这种老的env_id和新的env_id都能通过的情况。

Thinking 4.3

Q:

思考下面的问题，并对这个问题谈谈你的理解：请回顾 kern/env.c 文件中 mkenv_id() 函数的实现，该函数不会返回 0，请结合系统调用和 IPC 部分的实现与env_id2env() 函数的行为进行解释。

A:

从实现上讲在mkenv_id中，env_id高22位不可能为0，故env_id不可能为0。从理由上讲，在实现系统调用和IPC部分时，通过env_id是否为0来判断是否是当前进程，这有利于区分是否是当前进程。

Thinking 4.4

Q:

关于 fork 函数的两个返回值，下面说法正确的是：

- A、fork 在父进程中被调用两次，产生两个返回值
- B、fork 在两个进程中分别被调用一次，产生两个不同的返回值
- C、fork 只在父进程中被调用了一次，在两个进程中各产生一个返回值
- D、fork 只在子进程中被调用了一次，在两个进程中各产生一个返回值

A:

C

Thinking 4.5

Q:

上述“最小版”策略要求你在遍历时跳过UXSTACK与UVPT/VPT/VPD等特殊区域。请结合kern/env.c 中 env_init 的映射行为、include/mmu.h 的内存布局图，解释：

- 为什么子进程的UXSTACK必须重新分配，而不能共享或设为COW？
- 为什么不应应对UVPT/VPT/VPD等自映射区域调用duppage？如果误处理，可能引发哪些不一致或错误行为？

A:

- UXSTACK用于处理用户态的页写入异常，特别是COW页面的复制处理。如果父子两共用一个栈，则当父子两共同发生页写入异常时，会在同一栈上交错写入，导致数据地址混乱，最终崩溃。设为 COW 也不可行，因为在 COW 处理过程中异常栈本身可能被写入，若异常栈也是 COW 的，会触发递归的页写入异常，导致无限循环。
- UVPT是页表自映射的区域，反应了页表的二级结构。如果调用duppage，子进程读取UVPT时将会读取到父亲的页表，这样做不仅破坏了进程独立的地址空间，还可能导致父进程页表被子进程页表修改等一些列不可预知的情况。

Thinking 4.6

Q:

在遍历地址空间存取页表项时你需要使用到 vpd和 vpt这两个指针，请参考user/include/lib.h 中的相关定义，思考并回答这几个问题：

- vpt 和 vpd 的作用是什么？怎样使用它们？
- 从实现的角度谈一下为什么进程能够通过这种方式来存取自身的页表？
- 它们是如何体现自映射设计的？
- 进程能够通过这种方式来修改自己的页表项吗？

A:

- vpt和vpd的作用分别是直接访问页表项和页目录项。通过vpt+vpn(虚拟页号)访问页表项，通过vpd+vpn的高十位访问。
- vpt和vpd分别指向了两个值——UVPT和UVPT + (PDX(UVPT) << PGSHIFT)，这两个值分别是用户地址空间中第0张页表的首地址和页目录的首地址。所以我们可以直接通过vpt和vpd访问到用户进程页表和页目录。
- vpt和vpd都能访问页目录，在这里页目录也只是1024张页表中的一张普通的页表。体现了自映射机制。

- 不能，暴露在UVPT的页表时是只读的，修改页表只能进入内核

Thinking 4.7

Q: 在 `do_tlb_mod` 函数中，你可能注意到了一个向异常处理栈复制 `Trapframe`运行现场的过程，请思考并回答这几个问题：

- 这里实现了一个支持类似于“异常重入”的机制，而在什么时候会出现这种“异常重入”？
- 内核为什么需要将异常的现场 `Trapframe`复制到用户空间？

A:

- 在异常处理函数中重新引发异常，就会出现异常重入。例如，在异常处理函数中，访问了一个COW页，这样就会引发异常重入。
- 异常的处理其实是在用户态实现的，用户态无法访问内核空间，因此需要复制到用户空间。

Thinking 4.8

Q:

在用户态处理页写入异常，相比于在内核态处理有什么优势？

A:

体现了微内核的思想，将大部分功能放到用户态中，内核仅保留必要的功能，这样用户态的功能崩溃了也不会导致内核崩溃，仅更有利于操作系统的移植。

Thinking 4.9

Q:

为什么在设置写时复制保护机制之前，需通过 `syscall_set_tlb_mod_entry`设置父进程页写入异常处理函数？若在写时复制保护机制完成之后再设置会怎样？

A:

- `duppage`遍历父进程地址空间，将所有可写页面的页表项加上 `PTE_COW` 标志并清除 `PTE_D` 位，一旦某个页面被设为 `COW`，对该页面的任何写入都会触发 `TLB Mod` 异常。如果此时异常处理函数（`env_user_tlb_mod_entry`）尚未设置，内核 `do_tlb_mod` 跳转的是未定义地址或空指针，会导致进程崩溃。
- `duppage` 过程中，父进程自身仍在运行，可能对已设为 `COW` 的页面执行写入操作（如修改局部变量、压栈等）。若此时异常处理函数未注册，一旦触发 `TLB Mod` 异常，内核无法正确处理，直接 `panic` 或跳转到非法地址。特别是栈页面（`stack`）也是 `COW` 目标之一，函数调用（压栈返回地址）本身就会触发写入，因此危险性极高。

难点分析

个人认为难点主要集中在以下4个方面：

1. 页写入异常的处理：页写入异常是唯一在用户态处理的异常，因此在异常处理过程中可能发生中断，切换进程等一系列不可预知的结果。且处理过程中频繁切换用户态和内核态，中途经过的函数较多，且频繁修改栈和返回地址，极易出错。

2. 理清系统调用和平常中断的区别，了解两者返回路径的不同。
3. fork过程中子进程的创建以及将子进程的地址映射到和父进程同一块的虚拟地址空间
4. IPC的实现，返回单个值和交流一整个物理页的实现

心得体会

lab4的内容还是比较简单的，毕竟只是在lab3的基础上做了一些拓展并没有搞出一些新的花样，通过学习lab4，我对系统调用和异常处理流程有了一个更清晰的认识，同时用户态的异常处理函数也让我接触到了微内核的思想，但是我个人还是觉得微内核并不够好，实现起来还是太麻烦了，且会带来更大的性能开销。